Встраивание вычислений в PostgreSQL: PL*, extensions, а теперь и WASM

Дмитрий Дорофеев

PL*

Programming Language SQL

- PL/SQL Появился в Oracle V5.1 1986 год
- PL/SQL Реально заработал в Oracle V6 1988 год
- PL/pgSQL Появился в PostgreSQL 6.4 1998 год

- Полноценный язык программирования: циклы, условия, переменные и т.д.
- Поддержка исключений
- Тесная интеграция с SQL
- За основу был взят язык Ada

PL/Python - untrusted

- Впервые добавлен в PostgreSQL 7.3 (2002 год)
- Python2: не поддерживается с PostgreSQL 13 (2020 год)
- Python3: plpython3u появился в PostgreSQL 9.1 (2011 год)
- Можно установить и использовать модули Python
- Доступно в стандартном дистрибутиве PostgreSQL
- https://madlib.apache.org/

PL/Tcl и PL/Perl

- Trusted и Untrasted
- Доступны в стандартном дистрибутиве PostgreSQL
- Но при этом, ничего значимого на этих языках не написано

PL/Java

- Создаём jar (пишем код с аннотациями)
- Помещаем jar на сервер PostgreSQL
- Запускаем `select sqlj.install_jar()` (внутри запакованы CREATE FUNCTION)
- Выставляем `select sqlj.set_classpath('public', 'myjar')`
- Можно пользоваться функциями ;-)

PL/dotnet

- C#
- F#

- Самый быстрый PL (по мнению разработчиков :)
- Можно писать CREATE FUNCTION и инлайн код на (C|F)#

PL/v8

- **JavaScript**
- Trusted Only
- Пакеты удалили из всех репозиториев (спасибо, Google)
- Хорошая поддержка типов в PostgreSQL

Экзотика

- Haskell
- Lua
- PHP
- R
- Rust
- Shell
- XSLT

https://wiki.postgresql.org/wiki/PL_Matrix

Расширения

- pg_financial
- MADlib

- Не особо развиваются, в отличие от PostGIS!
- Всегда можно написать своё расширение.

Собственное расширение

- Рекомендую pgrx:
- Лёгкий запуск любой версии PostgreSQL
- Ручные и автотесты
- Безопасные Rust-обёртки над PostgreSQL API
- Можно всегда дописать код на Си

WASM

- Абстракция выполнимого кода
- Неудачные попытки:
  - Java (Write Once – Debug Everywhere)
  - Docker (If WASM+WASI existed in 2008, we wouldn't have needed to create Docker. That's how important it is. WebAssembly on the server is the future of computing. - Solomon Hykes co-founder of Docker)
- История повторяется как и с Java: придумали для Web, а используется на сервере!

WASM и PostgreSQL

- https://github.com/wasmerio/wasmer-postgres (в упадке)
- https://github.com/dylibso/pg_extism (на базе plv8)
- https://github.com/supabase/wrappers (SDK для написания FDW на WASM, но не только)

- Неожиданный эффект: https://github.com/electric-sql/pglite

PostgreSQL, скомпилированный в WASM с поддержкой расширений!

Ещё один способ: plv8

- Движок v8 нативно умеет запускать WASM
- Воспользуемся этим!
- plv8 в помощь!

Swagger спецификация для API Luxms BI

- **Цель:** генерация Swagger спецификаций на основе данных, хранящихся в таблице
- Воспользуемся DSL oxlip
- https://github.com/oxlip-lang/oal
  - Написан на Rust
  - Компилируется в WASM и пакуется с помощью wasm-pack

Конвейер

Таблица

- Статус,
- Шаблон URL,
- Заголовки,
- Примеры запросов,
- Примеры ответов и т.д.

INSERT
UPDATE
DELETE

Trigger

- Создаём скрипт на oxlip
- и запускаем генератор Swagger спеки на WASM

Rust

```rust
use wasm_bindgen::prelude::*;

#[wasm_bindgen]
pub fn greet(name: &str) -> String {
    format!("Hello, {}!", name)
}
```

Сборка

```
wasm-pack build --target bundler
wasm-pack build --target nodejs
```

вы получите в папке pkg следующие файлы:

- `my_crate_bg.wasm` — оптимизированный WASM
- `my_crate.js` — JS-обёртка, сгенерированная wasm-bindgen
- `my_crate.d.ts` — TypeScript-декларации
- `package.json` — метаданные для npm

Запуск

```
import * as wasm from 'my-crate';

wasm.greet("World");

// "Hello, World!"
```

PostgreSQL

- WASM файл кладём в таблицу в BYTEA
- Дописываем обертку для запуска WASM
- Готово!

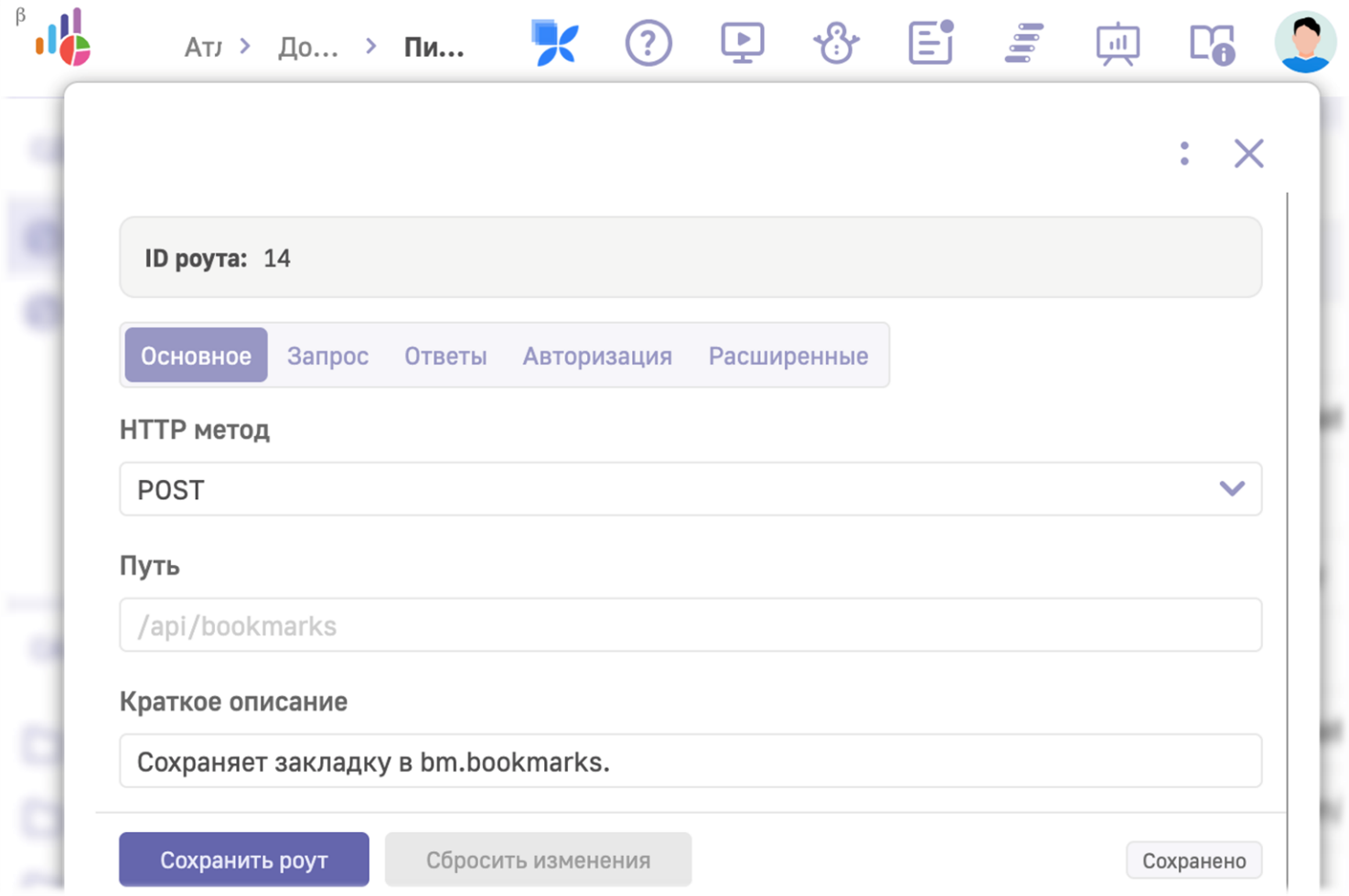
Запуск

```
CREATE OR REPLACE FUNCTION utils.oal_compile(_code TEXT)
RETURNS TEXT
LANGUAGE plv8
AS $body$

const response = plv8.execute(
    `select content from mytable where alt_id = ?`,
    ['oal_wasm_bg.wasm']
);
const bytes = new Uint8Array(response[0].content);

const wasmModule = new WebAssembly.Module(bytes);
const wasmInstance = new WebAssembly.Instance(wasmModule, imports);
return wasmInstance.exports.compile(_code);
$body$;
```

Бонус: Онлайн редактор для написания документации



Swagger spec создаётся на-лету!

Сохранить роут | Сбросить изменения | Сохранено

Swagger | YAML | OXLIP | Diff

Servers

/

Authorize

POST /api/bookmarks Сохраняет закладку в bm.bookmarks.

Сохраняет закладку в bm.bookmarks.

Parameters

Cancel

Конвейер 2

Браузер

- Статус,
- Шаблон URL,
- Заголовки,
- Примеры запросов,
- Примеры ответов и т.д.

UI Events

Callback

- Создаём скрипт на oxlip
- и запускаем генератор Swagger спеки на WASM

Это тот же самый WASM файл, что и в базе!

Генератор кода на oxlip – тот же код на JS, что и в базе

Luxms BI

Luxms BI

Создаём умные решения для умного управления.

luxmsbi.ru

sales@luxmsbi.ru

канал

комьюнити

Аналитика, которой можно доверять.

Решения, которые работают.